HCL Assessment

CVV ANAND,
23BTRCL176,
HCLTFP2297504,
AIML.

SECTION A: ADVANCED SQL || Maximum Marks: 20

Question 1: Database Design and Advanced Queries (8 Marks)

Create a database named EnterpriseDB.
Create the following tables.
Requirements:

- Define appropriate data types and constraints.
- Set primary keys and an appropriate foreign key.
- Insert at least 15 employee records and five department records.
- Include employees with different salaries, departments, and managers.
- Include at least one department without employees.

```sql
CREATE DATABASE EnterpriseDB;

USE EnterpriseDB;

CREATE TABLE Departments (
    DepartmentID VARCHAR(10) PRIMARY KEY,
    DepartmentName VARCHAR(50) NOT NULL,
    Location VARCHAR(50)
);

CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50) NOT NULL,
    DepartmentID VARCHAR(10),
    Salary DECIMAL(10,2) NOT NULL,
    JoiningDate DATE,
    ManagerID INT,
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID),
    FOREIGN KEY (ManagerID) REFERENCES Employees(EmployeeID)
);
```

```
22    •INSERT INTO Departments VALUES
23     ('D01', 'Engineering', 'Mumbai'),
24     ('D02', 'Marketing', 'Delhi'),
25     ('D03', 'Finance', 'Bengaluru'),
26     ('D04', 'Human Resources', 'Hyderabad'),
27     ('D05', 'Research & Development', 'Pune');
28
29    •INSERT INTO Employees VALUES
30     (1, 'Amit Sharma', 'D01', 55000.00, '2023-01-15', NULL),
31     (2, 'Priya Patel', 'D01', 48000.00, '2026-02-20', 1),
32     (3, 'Rahul Verma', 'D02', 62000.00, '2022-11-05', NULL),
33     (4, 'Sneha Gupta', 'D02', 51000.00, '2023-03-10', 3),
34     (5, 'Vikram Singh', 'D02', 47000.00, '2023-04-18', 3),
35     (6, 'Ananya Iyer', 'D03', 70000.00, '2022-08-22', NULL),
36     (7, 'Karthik Rao', 'D03', 58000.00, '2023-01-30', 6),
37     (8, 'Divya Nair', 'D03', 53000.00, '2025-05-01', 6),
38     (9, 'Arjun Mehta', 'D04', 45000.00, '2023-06-12', NULL),
39     (10, 'Pooja Reddy', 'D04', 42000.00, '2023-07-05', 9),
40     (11, 'Suresh Kumar', 'D01', 49000.00, '2023-08-14', 1),
41     (12, 'Neha Joshi', 'D02', 54000.00, '2023-09-01', 3),
42     (13, 'Ravi Kumar', 'D04', 44000.00, '2024-09-20', 9),
43     (14, 'Meera Krishnan', 'D03', 61000.00, '2023-10-01', 6),
44     (15, 'Deepak Saxena', 'D01', 52000.00, '2023-10-15', 1);
45
```

a. Display employee names, department names, and manager names using joins. **(2 marks)**

```
48
49    •SELECT e.EmployeeName, d.DepartmentName, e.ManagerID
50     FROM Employees e JOIN Departments d ON e.DepartmentID=d.DepartmentID;
51
```

Output | Result #9

EmployeeName	DepartmentName	ManagerID
Amit Sharma	Engineering	NULL
Priya Patel	Engineering	1
Suresh Kumar	Engineering	1
Deepak Saxena	Engineering	1
Rahul Verma	Marketing	NULL
Sneha Gupta	Marketing	3
Vikram Singh	Marketing	3
Neha Joshi	Marketing	3
Ananya Iyer	Finance	NULL
Karthik Rao	Finance	6
Divya Nair	Finance	6
Meera Krishnan	Finance	6

OK. 15 records retrieved in 2.288ms

a. Find employees earning more than their department's average salary. **(2 marks)**

```
51
52  SELECT EmployeeName, DepartmentID, Salary
53  FROM Employees e
54  WHERE Salary > (SELECT AVG(Salary)
55  FROM Employees
56  WHERE DepartmentID = e.DepartmentID);
57
```

Output | Result #10

EmployeeName	DepartmentID	Salary
Amit Sharma	D01	55000.00
Rahul Verma	D02	62000.00
Ananya Iyer	D03	70000.00
Arjun Mehta	D04	45000.00
Neha Joshi	D02	54000.00
Ravi Kumar	D04	44000.00
Meera Krishnan	D03	61000.00
Deepak Saxena	D01	52000.00

a. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST. **(2 marks)**

```
58  SELECT MAX(Salary) FROM Employees
59  WHERE Salary < (SELECT MAX(Salary) FROM Employees);
```

Output | Result #11

MAX(Salary)
62000.00

Question 2:
Create a Sales table with the following columns:

- SaleID
- CustomerID
- SaleDate
- ProductName
- Quantity
- UnitPrice

```sql
CREATE TABLE Sales (
    SaleID INT PRIMARY KEY,
    CustomerID INT,
    SaleDate DATE,
    ProductName VARCHAR(100),
    Quantity INT,
    UnitePrice DECIMAL (10,2)
);
INSERT INTO Sales VALUES
(11, 101, '2024-01-05', 'Laptop', 2, 45000.00),
(12, 102, '2024-01-12', 'Mouse', 5, 800.00),
(13, 103, '2024-02-01', 'Keyboard', 3, 1500.00),
(14, 101, '2024-02-10', 'Monitor', 1, 12000.00),
(15, 104, '2024-03-03', 'Printer', 1, 8500.00),
(16, 102, '2024-03-15', 'Laptop', 1, 45000.00),
(17, 105, '2024-04-01', 'Headphones', 4, 2200.00),
(18, 103, '2024-04-20', 'USB Cable', 10, 300.00),
(19, 101, '2024-05-05', 'Webcam', 2, 3500.00),
(20, 106, '2024-05-18', 'Laptop', 1, 52000.00),
(21, 104, '2024-06-02', 'Mouse', 3, 900.00),
(22, 102, '2024-06-25', 'Keyboard', 2, 1800.00),
(23, 105, '2024-07-10', 'Monitor', 2, 13000.00),
(24, 103, '2024-08-01', 'Printer', 1, 9000.00),
(25, 106, '2024-08-20', 'Headphones', 1, 2500.00);
```

Insert at least 15 sample records covering multiple customers, products, and dates.
Write SQL queries to:

a. Calculate total revenue for each product. **(2 marks)**

```sql
SELECT ProductName, SUM(Quantity * UnitePrice) AS TotalRevenue
FROM Sales
GROUP BY ProductName;
```

Output | Result #15

ProductName	TotalRevenue
Laptop	187000.00
Mouse	6700.00
Keyboard	8100.00
Monitor	38000.00
Printer	17500.00
Headphones	11300.00
USB Cable	3000.00
Webcam	7000.00

a. Calculate monthly revenue and month-over-month revenue change using a window function. **(2 marks)**

```
94    SELECT * FROM Sales WHERE SaleDate >= '2024-05-01' AND SaleDate < '2024-06-01';
```

Output | Result #16

SaleID	CustomerID	SaleDate	ProductName	Quantity	UnitePrice
19	101	05/05/2024	Webcam	2	3500.00
20	106	05/18/2024	Laptop	1	52000.00

c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better. **(2 marks)**
SELECT *
FROM Employees
WHERE YEAR(JoiningDate) = 2025;
Assume JoiningDate may be indexed.

```
96    SELECT *
97    FROM Employees
98    WHERE JoiningDate >= '2025-01-01' AND JoiningDate < '2026-01-01';
```

Output | Result #17

EmployeeID	EmployeeName	DepartmentID	Salary	JoiningDate	ManagerID
8	Divya Nair	D03	53000.00	05/01/2025	6

Question 3: SQL Troubleshooting (6 Marks)
A company maintains millions of employee and transaction records. Its reporting queries are running slowly.
Perform the following tasks:
a. Explain how an execution plan helps identify expensive operations in a query. **(2 marks)**
An execution plan shows how a database runs a query. It helps identify slow operations, such as table scans and joins, so we can improve query performance using indexes and optimization.

b. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning. **(2 marks)**
Create a composite index on DepartmentID and Salary because queries filter employees using both columns. This index helps the database find matching employees faster and improves query performance**.**

c. Write a query to identify duplicate customer email addresses and display only addresses occurring more than once. **(2 marks)**

```sql
SELECT Email, COUNT(*) AS Times
FROM Customers
GROUP BY Email
HAVING COUNT(*) > 1;
```

SECTION B: LINUX || Maximum Marks: 15

Question 4: Log Analysis (5 Marks)

Assume /var/log/application.log contains millions of log entries in the following format:
2026-10-28 10:15:23 ERROR payment-service Transaction failed
Write Linux commands to:

a. Find all lines containing ERROR. **(1 mark)**
b. Display the 20 most recent error entries in the file. **(1 mark)**
c. Count the total number of error entries. **(1 mark)**
d. Extract error entries for a specified date into a separate file. **(1 mark)**
e. Explain how you would calculate the number of errors occurring in each hour. **(1 mark)**

```
cvvanand@AnandCVV:~$ grep "ERROR" /var/log/application.log
grep: /var/log/application.log: No such file or directory
cvvanand@AnandCVV:~$ sudo mkdir -p /var/log
[sudo: authenticate] Password:
cvvanand@AnandCVV:~$ sudo nano /var/log/application.log
cvvanand@AnandCVV:~$ grep "ERROR" /var/log/application.log
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:45 ERROR auth-service Login failed
2026-10-28 11:30:12 ERROR payment-service Payment timeout
2026-10-27 14:20:00 ERROR database-service Connection failed
2026-10-28 12:10:30 ERROR payment-service Transaction declined
cvvanand@AnandCVV:~$ grep "ERROR" /var/log/application.log | tail -n 20
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:45 ERROR auth-service Login failed
2026-10-28 11:30:12 ERROR payment-service Payment timeout
2026-10-27 14:20:00 ERROR database-service Connection failed
2026-10-28 12:10:30 ERROR payment-service Transaction declined
cvvanand@AnandCVV:~$ grep -c "ERROR" /var/log/application.log
5
cvvanand@AnandCVV:~$ grep "^2026-10-28.*ERROR" /var/log/application.log > ~/errors_2026-10-28.log
cvvanand@AnandCVV:~$ cat ~/errors_2026-10-28.log
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:45 ERROR auth-service Login failed
2026-10-28 11:30:12 ERROR payment-service Payment timeout
2026-10-28 12:10:30 ERROR payment-service Transaction declined
cvvanand@AnandCVV:~$ awk '$3 == "ERROR" {count[substr($2,1,2)]++} END {for (hour in count) print hour ":00 -", count[hour], "errors"}' /var/log/application.
log
10:00 - 1 errors
11:00 - 2 errors
12:00 - 1 errors
14:00 - 1 errors
cvvanand@AnandCVV:~$
```

Question 5: Permissions and System Troubleshooting (5 Marks)

A Linux server hosts an application that processes confidential financial reports.
Perform the following tasks:

a. Create a group named finance and a report file, assuming you have the required privileges. **(1 mark)**
b. Set permissions so the owner can read and write, the finance group can read, and other users have no access. **(1 mark)**
c. Identify the processes consuming the most CPU. **(1 mark)**
d. Display memory usage and filesystem disk utilization. **(1 mark)**
e. Find files larger than 100 MB under a specified application directory. **(1 mark)**

```
cvvanand@AnandCVV:~$ sudo groupadd finance
[sudo: authenticate] Password:
cvvanand@AnandCVV:~$ touch report.txt
cvvanand@AnandCVV:~$ sudo chown $USER:finance report.txt
cvvanand@AnandCVV:~$ chmod 640 report.txt
cvvanand@AnandCVV:~$ top
top - 06:47:54 up 26 min,  2 users,  load average: 0.00, 0.00, 0.00
Tasks:  28 total,   1 running,  27 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7776.6 total,   7171.0 free,    548.3 used,    208.9 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7228.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   1089 cvvanand  20   0    8288   6080   3916 R   0.7   0.1   0:00.06 top
      1 root      20   0   24116  15428  11624 S   0.0   0.2   0:00.63 systemd
      2 root      20   0    3180   2208   2072 S   0.0   0.0   0:00.00 init-systemd(Ub
      6 root      20   0    3180   2048   1940 S   0.0   0.0   0:00.00 init
     46 root      19  -1   50416  16908  15616 S   0.0   0.2   0:00.18 systemd-journal
     78 systemd+  20   0   22540  14536  11916 S   0.0   0.2   0:00.06 systemd-resolve
     85 root      20   0   35428  12336   9176 S   0.0   0.2   0:00.46 systemd-udevd
    104 root      20   0    2888   1944   1816 S   0.0   0.0   0:00.04 chronyd-starter
    105 root      20   0    4472   3140   2888 S   0.0   0.0   0:00.00 cron
    106 message+  20   0    8820   5320   4628 S   0.0   0.1   0:00.06 dbus-daemon
    110 root      20   0   44092  29740  14604 S   0.0   0.4   0:00.21 networkd-dispat
    127 root      20   0   18588   9484   8200 S   0.0   0.1   0:00.08 systemd-logind
    182 syslog    20   0  220548   5596   4656 S   0.0   0.1   0:00.07 rsyslogd
    204 _chrony   20   0   20420   7660   6864 S   0.0   0.1   0:00.06 chronyd
    214 _chrony   20   0   12092   2448   1804 S   0.0   0.0   0:00.00 chronyd
    229 root      20   0    5492   2716   2532 S   0.0   0.0   0:00.00 agetty
    234 root      20   0  123456  32540  17932 S   0.0   0.4   0:00.12 unattended-upgr
    317 root      20   0    8500   5448   4620 S   0.0   0.1   0:00.01 login
    360 root      20   0   22332  13624  11004 S   0.0   0.2   0:00.22 systemd
    362 root      20   0   22912   4000   2012 S   0.0   0.1   0:00.00 (sd-pam)
    390 root      20   0    6136   5368   3852 S   0.0   0.1   0:00.04 bash
    503 root      20   0    8644   5596   4756 S   0.0   0.1   0:00.00 login
    534 cvvanand  20   0   22340  13552  10928 S   0.0   0.2   0:00.07 systemd
    536 cvvanand  20   0   22908   4140   2140 S   0.0   0.1   0:00.00 (sd-pam)
    560 cvvanand  20   0    6136   5412   3860 S   0.0   0.1   0:00.02 bash
    998 root      20   0    3196   1124    980 S   0.0   0.0   0:00.00 SessionLeader
    999 root      20   0    3196   1260   1108 S   0.0   0.0   0:00.00 Relay(1005)
   1005 cvvanand  20   0    6268   5608   3936 S   0.0   0.1   0:00.03 bash

cvvanand@AnandCVV:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.6Gi       545Mi       7.0Gi       3.5Mi       208Mi       7.1Gi
Swap:          2.0Gi          0B       2.0Gi
cvvanand@AnandCVV:~$ find /opt/myapp -type f -size +100M
find: '/opt/myapp': No such file or directory
cvvanand@AnandCVV:~$ pwd
/home/cvvanand
```

Question 6: Bash Scripting (5 Marks)

Write a Bash script named backup_check.sh that:

- Accepts source and destination directories as command-line arguments.
- Validates that both arguments are provided.
- Checks whether the source directory exists.
- Creates the destination directory if it does not exist.
- Creates a timestamped compressed archive of the source directory.
- Displays a success message if the backup succeeds.
- Displays an appropriate error message if the backup fails.

Expected Output: Submit the complete script and demonstrate its execution using valid and invalid arguments.

Evaluation:

- Argument validation and directory checks: 2 marks
- Archive creation: 2 marks
- Error handling: 1 mark

```
chmod +x backup_check.sh
```

```
mkdir -p source_data
echo "Sample backup data" > source_data/data.txt

# Valid arguments
./backup_check.sh source_data backup_folder

# Invalid: missing arguments
./backup_check.sh

# Invalid: source directory does not exist
./backup_check.sh missing_folder backup_folder
```

```
cvvanand@AnandCVV:~$ mkdir backup_practice
mkdir: backup_practice: File exists
cvvanand@AnandCVV:~$ cd backup_practice
cvvanand@AnandCVV:~/backup_practice$ nano backup_check.sh
cvvanand@AnandCVV:~/backup_practice$ chmod +x backup_check.sh
cvvanand@AnandCVV:~/backup_practice$ mkdir -p source_data
cvvanand@AnandCVV:~/backup_practice$ echo "This is my sample backup file" > source_data/data.txt
cvvanand@AnandCVV:~/backup_practice$ ./backup_check.sh source_data backup_folder
Backup successful: backup_folder/backup_20261028_150449.tar.gz
cvvanand@AnandCVV:~/backup_practice$ ls -l backup_folder
total 4
-rw-r--r-- 1 cvvanand cvvanand 189 Oct 28 15:04 backup_20261028_150449.tar.gz
cvvanand@AnandCVV:~/backup_practice$ ./backup_check.sh
Error: Please provide source and destination directories.
Usage: ./backup_check.sh <source_directory> <destination_directory>
cvvanand@AnandCVV:~/backup_practice$ ./backup_check.sh missing_folder backup_folder
Error: Source directory does not exist.
cvvanand@AnandCVV:~/backup_practice$ ./backup_check.sh source_data
Error: Please provide source and destination directories.
Usage: ./backup_check.sh <source_directory> <destination_directory>
cvvanand@AnandCVV:~/backup_practice$ tar -tzf backup_folder/backup_*.tar.gz
source_data/
source_data/data.txt
cvvanand@AnandCVV:~/backup_practice$
```

SECTION C: PYSPARK DATA ENGINEERING || Maximum Marks: 20

Question 7: PySpark DataFrame Operations (6 Marks)

Create a PySpark DataFrame containing the following columns:

- EmployeeID
- EmployeeName
- Department
- Salary
- JoiningDate

Insert at least 15 sample records.

```python
import os
import sys

# Fix: Configure Spark to use the current Python interpreter
os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count, round

# Create Spark Session
spark = (
    SparkSession.builder
    .appName("EmployeeDataFrameOperations")
    .master("local[*]")
    .getOrCreate()
)

# Reduce unnecessary Spark log messages
spark.sparkContext.setLogLevel("ERROR")

# Sample employee data: 15 records
data = [
    (101, "Anand", "IT", 75000, "2023-01-15"),
    (102, "Rahul", "HR", 45000, "2022-03-10"),
    (103, "Priya", "Finance", 65000, "2021-07-20"),
    (104, "Sneha", "IT", 85000, "2020-11-05"),
    (105, "Kiran", "Sales", 55000, "2023-06-12"),
    (106, "Arjun", "IT", 95000, "2019-09-18"),
    (107, "Divya", "HR", 62000, "2022-08-25"),
    (108, "Ravi", "Finance", 72000, "2021-04-14"),
    (109, "Meena", "Sales", 48000, "2023-02-28"),
    (110, "Vijay", "IT", 68000, "2020-05-16"),
    (111, "Pooja", "HR", 58000, "2022-12-01"),
    (112, "Suresh", "Finance", 88000, "2018-10-22"),
    (113, "Neha", "Sales", 67000, "2021-01-30"),
    (114, "Ajay", "IT", 52000, "2024-02-11"),
    (115, "Lakshmi", "Finance", 78000, "2020-07-09")
]
```

Write and execute a PySpark program to:
a. Display the schema and first five records. **(1 mark)**

```
60   # (a) Display schema and first five records
61   print("\n========== (a) DATAFRAME SCHEMA ==========")
62   df.printSchema()
63
64   print("\n========== FIRST FIVE RECORDS ==========")
65   df.show(5, truncate=False)
66
```

Problems | Terminal | Output | Debug Console | Ports

```
========== (a) DATAFRAME SCHEMA ==========
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)


========== FIRST FIVE RECORDS ==========
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|101       |Anand       |IT        |75000 |2023-01-15 |
|102       |Rahul       |HR        |45000 |2022-03-10 |
|103       |Priya       |Finance   |65000 |2021-07-20 |
|104       |Sneha       |IT        |85000 |2020-11-05 |
|105       |Kiran       |Sales     |55000 |2023-06-12 |
+----------+------------+----------+------+-----------+
only showing top 5 rows
```

b. Filter employees whose monthly salary is greater than ₹60,000. **(1 mark)**

```
66
67   # (b) Filter employees whose monthly salary is greater than Rs. 60,000
68   print("\n========== (b) SALARY GREATER THAN 60000 ==========")
69
70   high_salary = df.filter(col("Salary") > 60000)
71   high_salary.show(truncate=False)
```

Problems | Terminal | Output | Debug Console | Ports

```
========== (b) SALARY GREATER THAN 60000 ==========
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|101       |Anand       |IT        |75000 |2023-01-15 |
|103       |Priya       |Finance   |65000 |2021-07-20 |
|104       |Sneha       |IT        |85000 |2020-11-05 |
|106       |Arjun       |IT        |95000 |2019-09-18 |
|107       |Divya       |HR        |62000 |2022-08-25 |
|108       |Ravi        |Finance   |72000 |2021-04-14 |
|110       |Vijay       |IT        |68000 |2020-05-16 |
|112       |Suresh      |Finance   |88000 |2018-10-22 |
|113       |Neha        |Sales     |67000 |2021-01-30 |
|115       |Lakshmi     |Finance   |78000 |2020-07-09 |
+----------+------------+----------+------+-----------+
```

c. Add an AnnualSalary column calculated from the monthly salary. **(1 mark)**

```
73  # (c) Add AnnualSalary column
74  print("\n========== (c) ANNUAL SALARY ==========")
75
76  df_annual = df.withColumn(
77      "AnnualSalary",
78      col("Salary") * 12
79  )
80
81  df_annual.show(truncate=False)
82
```

Problems | Terminal | Output | Debug Console | Ports

```
========== (c) ANNUAL SALARY ==========
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|101       |Anand       |IT        |75000 |2023-01-15 |900000      |
|102       |Rahul       |HR        |45000 |2022-03-10 |540000      |
|103       |Priya       |Finance   |65000 |2021-07-20 |780000      |
|104       |Sneha       |IT        |85000 |2020-11-05 |1020000     |
|105       |Kiran       |Sales     |55000 |2023-06-12 |660000      |
|106       |Arjun       |IT        |95000 |2019-09-18 |1140000     |
|107       |Divya       |HR        |62000 |2022-08-25 |744000      |
|108       |Ravi        |Finance   |72000 |2021-04-14 |864000      |
|109       |Meena       |Sales     |48000 |2023-02-28 |576000      |
|110       |Vijay       |IT        |68000 |2020-05-16 |816000      |
|111       |Pooja       |HR        |58000 |2022-12-01 |696000      |
|112       |Suresh      |Finance   |88000 |2018-10-22 |1056000     |
|113       |Neha        |Sales     |67000 |2021-01-30 |804000      |
|114       |Ajay        |IT        |52000 |2024-02-11 |624000      |
|115       |Lakshmi     |Finance   |78000 |2020-07-09 |936000      |
+----------+------------+----------+------+-----------+------------+
```

d. Group employees by department and calculate the average salary and employee count. **(2 marks)**

```python
# (d) Group employees by department
# Calculate average salary and employee count
print("\n========== (d) DEPARTMENT SUMMARY ==========")

department_summary = (
    df.groupBy("Department")
    .agg(
        round(avg("Salary"), 2).alias("AverageSalary"),
        count("EmployeeID").alias("EmployeeCount")
    )
    .orderBy("Department")
)

department_summary.show(truncate=False)

```

Problems | Terminal | Output | Debug Console | Ports

```
========== (d) DEPARTMENT SUMMARY ==========
+----------+-------------+-------------+
|Department|AverageSalary|EmployeeCount|
+----------+-------------+-------------+
|Finance   |75750.0      |4            |
|HR        |55000.0      |3            |
|IT        |75000.0      |5            |
|Sales     |56666.67     |3            |
+----------+-------------+-------------+
```

e. Display the five highest-paid employees. **(1 mark)**

```python
# (e) Display the five highest-paid employees
print("\n========== (e) TOP FIVE HIGHEST-PAID EMPLOYEES ==========")

top_five = (
    df.orderBy(col("Salary").desc())
    .limit(5)
)

top_five.show(truncate=False)

# Stop Spark Session
spark.stop()

print("\n========== PROGRAM EXECUTED SUCCESSFULLY ==========")

```

Problems | Terminal | Output | Debug Console | Ports

```
========== (e) TOP FIVE HIGHEST-PAID EMPLOYEES ==========
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|106       |Arjun       |IT        |95000 |2019-09-18 |
|112       |Suresh      |Finance   |88000 |2018-10-22 |
|104       |Sneha       |IT        |85000 |2020-11-05 |
|115       |Lakshmi     |Finance   |78000 |2020-07-09 |
|101       |Anand       |IT        |75000 |2023-01-15 |
+----------+------------+----------+------+-----------+


========== PROGRAM EXECUTED SUCCESSFULLY ==========
```

Expected Output: Submit the complete PySpark program and execution results.

Question 8: Data Cleaning and Aggregation (7 Marks)
A company receives a CSV file named transactions.csv containing:

- TransactionID
- CustomerID
- TransactionDate
- Product
- Quantity
- UnitPrice

The file contains duplicate records, missing values, and incorrectly formatted dates.
Write a PySpark program to:
a. Read the CSV file using an appropriate schema and header configuration. **(1 mark)**

```python
try:
    # (a) Read CSV using an appropriate schema and header
    base_dir = os.path.dirname(os.path.abspath(__file__))
    csv_path = os.path.join(base_dir, "transactions.csv")
    schema = StructType([
        StructField("TransactionID", StringType(), True),
        StructField("CustomerID", StringType(), True),
        StructField("TransactionDate", StringType(), True),
        StructField("Product", StringType(), True),
        StructField("Quantity", StringType(), True),
        StructField("UnitPrice", StringType(), True)
    ])
    df = (
        spark.read
        .option("header", True)
        .option("mode", "PERMISSIVE")
        .schema(schema)
        .csv(csv_path)
    )
    print("\n(a) ORIGINAL DATA")
    print("Original row count:", df.count())
    df.show(truncate=False)
```

Problems | Terminal | Output | Debug Console | Ports

```
(a) ORIGINAL DATA
Original row count: 9
+-------------+----------+---------------+--------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product |Quantity|UnitPrice|
+-------------+----------+---------------+--------+--------+---------+
|T001         |C101      |2026-10-01     |Laptop  |2.0     |50000.0  |
|T002         |C102      |02-10-2026     |Mouse   |5.0     |500.0    |
|T003         |C103      |2026-10-03     |Keyboard|3.0     |1500.0   |
|T001         |C101      |2026-10-01     |Laptop  |2.0     |50000.0  |
|T004         |C104      |invalid-date   |Monitor |2.0     |12000.0  |
|T005         |NULL      |2026-10-05     |Mouse   |4.0     |500.0    |
|T006         |C106      |2026-10-06     |Laptop  |NULL    |50000.0  |
|T007         |C107      |2026-10-07     |Keyboard|2.0     |NULL     |
|T008         |C108      |2026-10-08     |Monitor |1.0     |12000.0  |
+-------------+----------+---------------+--------+--------+---------+
```

b. Identify and remove duplicate transactions using a suitable business key. **(1 mark)**

```python
    # (b) Remove duplicate transactions using TransactionID
    df = df.dropDuplicates(["TransactionID"])

    print("\n(b) AFTER DUPLICATE REMOVAL")
    print("Rows remaining:", df.count())

    # Convert numeric columns safely
    df = (
        df.withColumn("Quantity", col("Quantity").cast(DoubleType()))
          .withColumn("UnitPrice", col("UnitPrice").cast(DoubleType()))
    )

```

Problems | Terminal | Output | Debug Console | Ports

```
+-------------+----------+---------------+--------+--------+---------+

(b) AFTER DUPLICATE REMOVAL
Rows remaining: 8
```

c. Handle missing values in required fields and explain your chosen approach. **(2 marks)**

```python
    # (c) Remove records missing required fields.
    # We drop incomplete records instead of inventing values because
    # IDs, dates, quantities, and prices are required for valid transactions.
    required_columns = [
        "TransactionID",
        "CustomerID",
        "TransactionDate",
        "Product",
        "Quantity",
        "UnitPrice"
    ]

    df = df.dropna(subset=required_columns)

    # Also remove empty or whitespace-only required text fields
    for name in ["TransactionID", "CustomerID", "Product"]:
        df = df.filter(col(name).rlike(r".*\S.*"))

    print("\n(c) AND (d) CLEANED DATA")
    print("Cleaned row count:", df.count())
    df.show(truncate=False)
```

Problems | Terminal | Output | Debug Console | Ports

```
Rows remaining: 8

(c) AND (d) CLEANED DATA
Cleaned row count: 4
+-------------+----------+---------------+--------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product |Quantity|UnitPrice|
+-------------+----------+---------------+--------+--------+---------+
|T001         |C101      |2026-10-01     |Laptop  |2.0     |50000.0  |
|T002         |C102      |2026-10-02     |Mouse   |5.0     |500.0    |
|T003         |C103      |2026-10-03     |Keyboard|3.0     |1500.0   |
|T008         |C108      |2026-10-08     |Monitor |1.0     |12000.0  |
+-------------+----------+---------------+--------+--------+---------+
```

d. Convert TransactionDate into a date type and handle invalid dates. **(1 mark)**

```
65    # (d) Convert dates and safely handle invalid formats
66    iso_date = try_to_timestamp(
67        col("TransactionDate"), lit("yyyy-MM-dd")
68    ).cast("date")
69
70    day_first_date = try_to_timestamp(
71        col("TransactionDate"), lit("dd-MM-yyyy")
72    ).cast("date")
73
74    df = df.withColumn(
75        "TransactionDate",
76        coalesce(iso_date, day_first_date)
77    )
```

Problems | Terminal | Output | Debug Console | Ports

```
Rows remaining: 8

(c) AND (d) CLEANED DATA
Cleaned row count: 4
+-------------+----------+---------------+--------+--------+----------+
|TransactionID|CustomerID|TransactionDate|Product |Quantity|UnitPrice|
+-------------+----------+---------------+--------+--------+----------+
|T001         |C101      |2026-10-01     |Laptop  |2.0     |50000.0   |
|T002         |C102      |2026-10-02     |Mouse   |5.0     |500.0     |
|T003         |C103      |2026-10-03     |Keyboard|3.0     |1500.0    |
|T008         |C108      |2026-10-08     |Monitor |1.0     |12000.0   |
+-------------+----------+---------------+--------+--------+----------+
```

e. Calculate revenue using Quantity × UnitPrice. **(1 mark)**

```python
    # (e) Calculate Revenue = Quantity * UnitPrice
    df = df.withColumn(
        "Revenue",
        col("Quantity") * col("UnitPrice")
    )

    print("\n(e) DATA WITH REVENUE")
    df.show(truncate=False)
```

Problems Terminal Output Debug Console Ports

```
|T008         |C108      |2026-10-08     |Monitor |1.0     |12000.0  |
+-------------+----------+---------------+--------+--------+---------+


(e) DATA WITH REVENUE
+-------------+----------+---------------+--------+--------+---------+---------+
|TransactionID|CustomerID|TransactionDate|Product |Quantity|UnitPrice|Revenue  |
+-------------+----------+---------------+--------+--------+---------+---------+
|T001         |C101      |2026-10-01     |Laptop  |2.0     |50000.0  |100000.0|
|T002         |C102      |2026-10-02     |Mouse   |5.0     |500.0    |2500.0   |
|T003         |C103      |2026-10-03     |Keyboard|3.0     |1500.0   |4500.0   |
|T008         |C108      |2026-10-08     |Monitor |1.0     |12000.0  |12000.0  |
+-------------+----------+---------------+--------+--------+---------+---------+
```

f. Calculate total revenue by product and display the results in descending order. **(1 mark)**

```python
    # (f) Total revenue by product in descending order
    product_revenue = (
        df.groupBy("Product")
        .agg(spark_sum("Revenue").alias("TotalRevenue"))
        .orderBy(col("TotalRevenue").desc())
    )

    print("\n(f) TOTAL REVENUE BY PRODUCT")
    product_revenue.show(truncate=False)

finally:
    # Stop Spark after completing all operations
    spark.stop()
```

Problems Terminal Output Debug Console Ports

```
(f) TOTAL REVENUE BY PRODUCT
+--------+------------+
|Product |TotalRevenue|
+--------+------------+
|Laptop  |100000.0    |
|Monitor |12000.0     |
|Keyboard|4500.0      |
|Mouse   |2500.0      |
+--------+------------+

SUCCESS: The process with PID 25788 (child process of PID 20284) has been terminated.
SUCCESS: The process with PID 20284 (child process of PID 33388) has been terminated.
SUCCESS: The process with PID 33388 (child process of PID 28432) has been terminated.
```

Expected Output: Submit the program and show the cleaned data and aggregation results.

Question 9: PySpark Joins and Performance (7 Marks)
A company maintains two datasets.
Customers

- CustomerID
- CustomerName
- City

Orders

- OrderID
- CustomerID
- OrderAmount
- OrderDate

Write a PySpark program to:
a. Load both datasets into DataFrames. **(1 mark)**

```
17  # (a) Load both CSV datasets into DataFrames
18  customers = spark.read.csv(
19      "customers.csv",
20      header=True,
21      inferSchema=True
22  )
23
24  orders = spark.read.csv(
25      "orders.csv",
26      header=True,
27      inferSchema=True
28  )
29
30  print("CUSTOMERS DATAFRAME")
31  customers.show()
32
33  print("ORDERS DATAFRAME")
34  orders.show()
35
```

Problems | Terminal | Output | Debug Console | Ports

```
To adjust logging level use sc.setLogLevel(newLevel). F
26/10/28 17:39:13 WARN NativeCodeLoader: Unable to load
CUSTOMERS DATAFRAME
+----------+------------+---------+
|CustomerID|CustomerName|     City|
+----------+------------+---------+
|         1|       Anand|Bengaluru|
|         2|        Ravi|Hyderabad|
|         3|        Sita|  Chennai|
|         4|       Priya|Bengaluru|
|         5|       Kiran|   Mumbai|
+----------+------------+---------+
```

ORDERS DATAFRAME

OrderID	CustomerID	OrderAmount	OrderDate
101	1	2500	2026-10-01
102	2	1500	2026-10-02
103	1	3000	2026-10-05
104	4	2000	2026-10-07

b. Join the datasets to display orders with customer details. **(2 marks)**

```python
# (b) Join datasets to display orders with customer details
print("ORDERS WITH CUSTOMER DETAILS")

customer_orders = customers.join(
    orders,
    on="CustomerID",
    how="inner"
)

customer_orders.select(
    "OrderID",
    "CustomerID",
    "CustomerName",
    "City",
    "OrderAmount",
    "OrderDate"
).show()

```

Problems | Terminal | Output | Debug Console | Ports

```
ORDERS WITH CUSTOMER DETAILS
+-------+----------+------------+---------+-----------+----------+
|OrderID|CustomerID|CustomerName|     City|OrderAmount| OrderDate|
+-------+----------+------------+---------+-----------+----------+
|    101|         1|       Anand|Bengaluru|       2500|2026-10-01|
|    102|         2|        Ravi|Hyderabad|       1500|2026-10-02|
|    103|         1|       Anand|Bengaluru|       3000|2026-10-05|
|    104|         4|       Priya|Bengaluru|       2000|2026-10-07|
+-------+----------+------------+---------+-----------+----------+
```

c. Identify customers who have never placed an order. **(1 mark)**

```python
# (c) Identify customers who never placed an order
print("CUSTOMERS WITHOUT ORDERS")

customers_without_orders = customers.join(
    orders,
    on="CustomerID",
    how="left_anti"
)

customers_without_orders.show()
```

Problems | Terminal | Output | Debug Console | Ports

```
CUSTOMERS WITHOUT ORDERS
+----------+------------+-------+
|CustomerID|CustomerName|   City|
+----------+------------+-------+
|         3|        Sita|Chennai|
|         5|       Kiran| Mumbai|
+----------+------------+-------+
```

d. Calculate the total order value by city. **(1 mark)**

```python
# (d) Calculate total order value by city
print("TOTAL ORDER VALUE BY CITY")

city_totals = customer_orders.groupBy(
    "City"
).agg(
    spark_sum("OrderAmount").alias("TotalOrderValue")
)

city_totals.show()
```

Problems | Terminal | Output | Debug Console | Ports

```
TOTAL ORDER VALUE BY CITY
+---------+---------------+
|     City|TotalOrderValue|
+---------+---------------+
|Bengaluru|           7500|
|Hyderabad|           1500|
+---------+---------------+
```

e. Explain when a broadcast join is useful and what conditions make it appropriate. **(1 mark)**
A broadcast join is useful when one DataFrame is small enough to be copied to all worker nodes, reducing data shuffling and improving join performance. It is appropriate when the smaller DataFrame fits comfortably in each worker's memory.

```
76  # (e) Example of a broadcast join
77  print("BROADCAST JOIN EXAMPLE")
78
79  broadcast_result = orders.join(
80      broadcast(customers),
81      on="CustomerID",
82      how="inner"
83  )
84
85  broadcast_result.select(
86      "OrderID",
87      "CustomerName",
88      "City",
89      "OrderAmount"
90  ).show()
```

Problems | Terminal | Output | Debug Console | Ports

```
BROADCAST JOIN EXAMPLE
+-------+------------+---------+-----------+
|OrderID|CustomerName|     City|OrderAmount|
+-------+------------+---------+-----------+
|    101|       Anand|Bengaluru|       2500|
|    102|        Ravi|Hyderabad|       1500|
|    103|       Anand|Bengaluru|       3000|
|    104|       Priya|Bengaluru|       2000|
+-------+------------+---------+-----------+
```

f. Explain the difference between transformations and actions, giving one example of each. **(1 mark)**

- Transformation: Creates a new Data Frame from an existing one. Example: filter ().
- Action: Executes the computation and produces a result. Example: show ().

```
92  # (f) Transformation and action examples
93  # Transformation: filter()
94  filtered_orders = orders.filter(
95      col("OrderAmount") > 2000
96  )
97
98  # Action: show()
99  print("ORDERS ABOVE 2000")
100 filtered_orders.show()
101
102 # Action: count()
103 print("TOTAL NUMBER OF ORDERS:", orders.count())
104
105 spark.stop()
106
```

Problems | Terminal | Output | Debug Console | Ports

```
+-------+----------+---------+----------+

ORDERS ABOVE 2000
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount| OrderDate|
+-------+----------+-----------+----------+
|    101|         1|       2500|2026-10-01|
|    103|         1|       3000|2026-10-05|
+-------+----------+-----------+----------+

TOTAL NUMBER OF ORDERS: 4
```

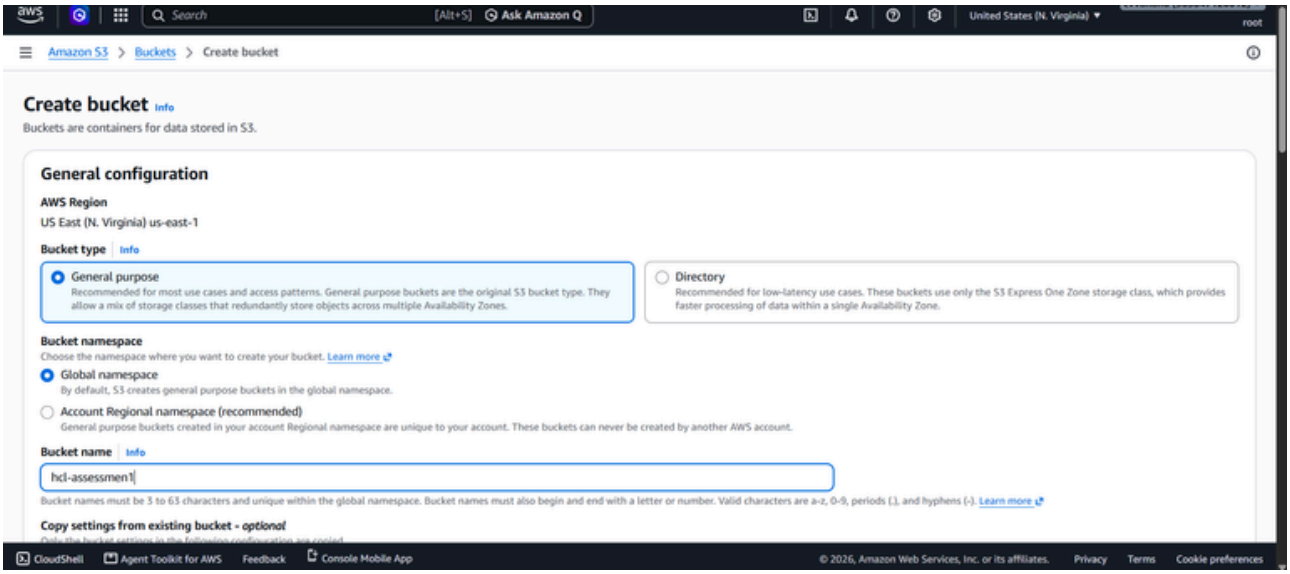
SECTION D: AWS CLOUD SERVICES || Maximum Marks: 15

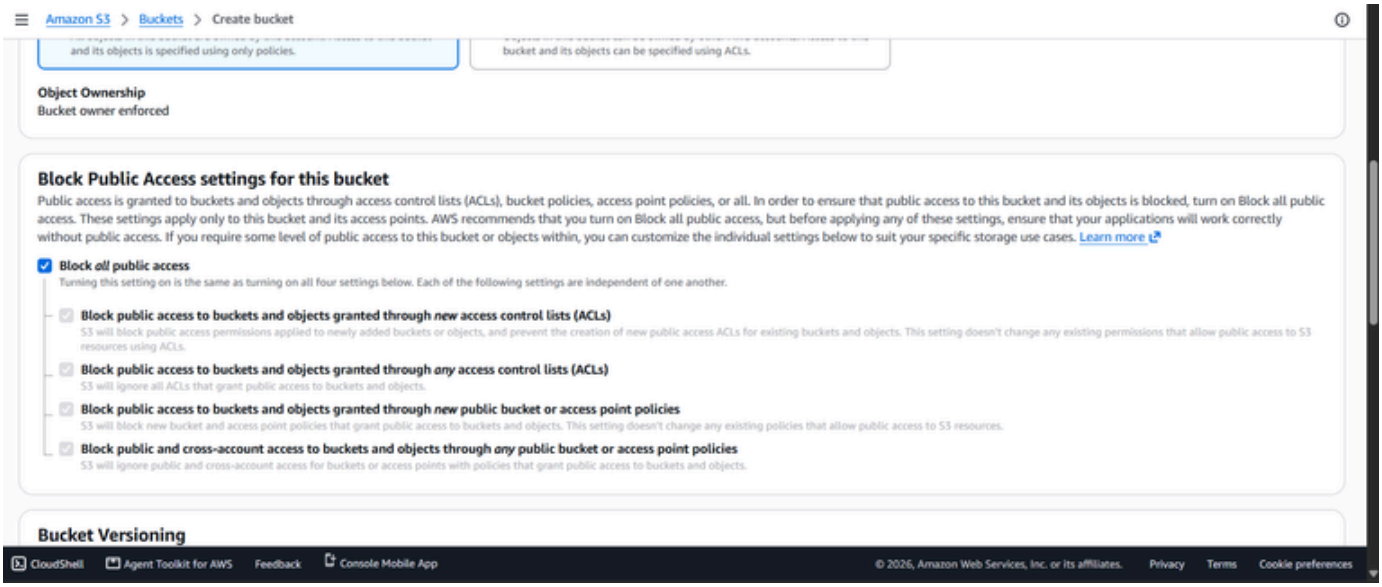
Question 10: S3 and IAM Security (5 Marks)

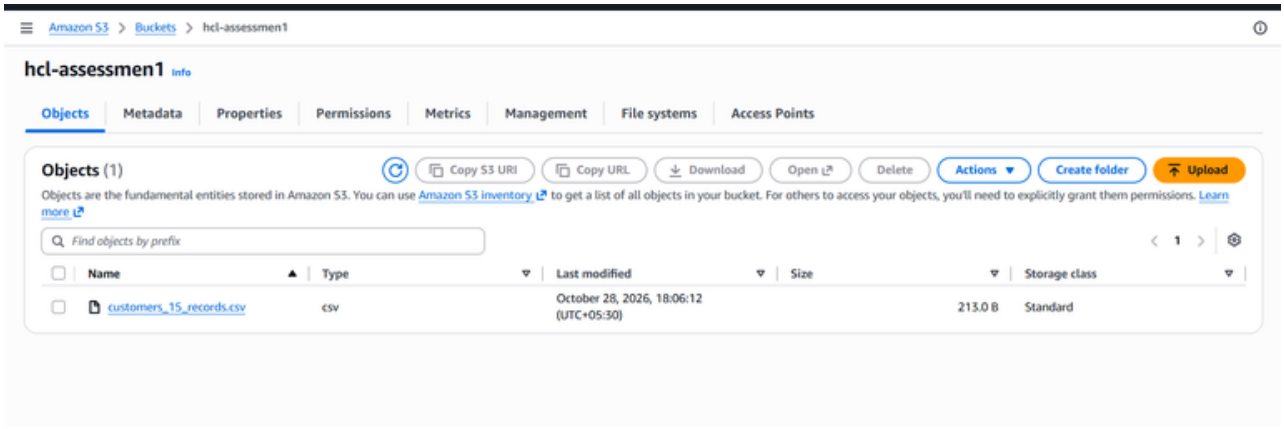
A company stores confidential employee reports in Amazon S3. Only authorized HR employees should be able to read these reports.

Perform or document the following:

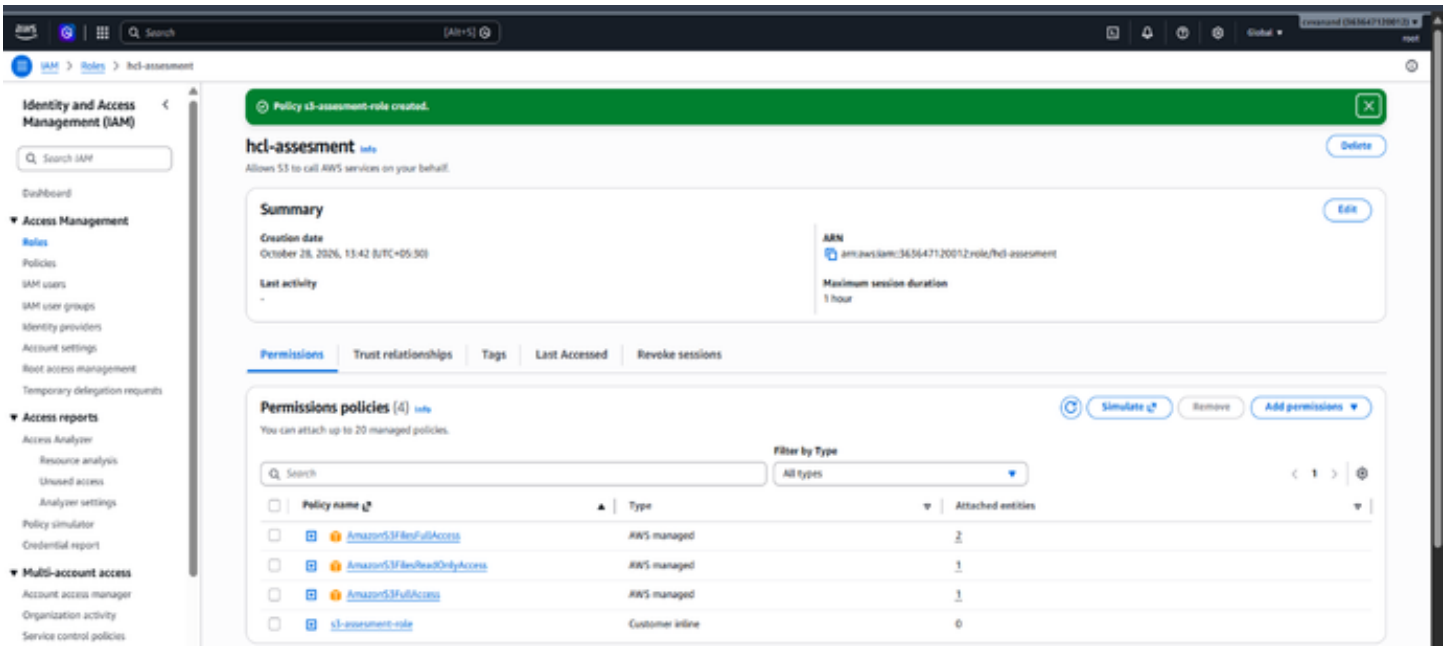
a. Create an S3 bucket with Block Public Access enabled and upload a sample file. **(1 mark)**







a. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. **(2 marks)**





Explanation:

- Effect: Allow grants permission.
- s3:GetObject allows users to read objects.
- Resource restricts access to objects inside the specified bucket.
- No permissions are granted to read objects in unrelated buckets.

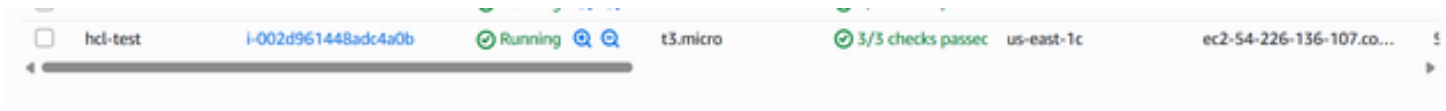
c. Explain how you would verify that an unauthorized user cannot read the file. **(1 mark)**
Attempt to access the file using an unauthorized IAM user and verify that AWS returns Access Denied (403).

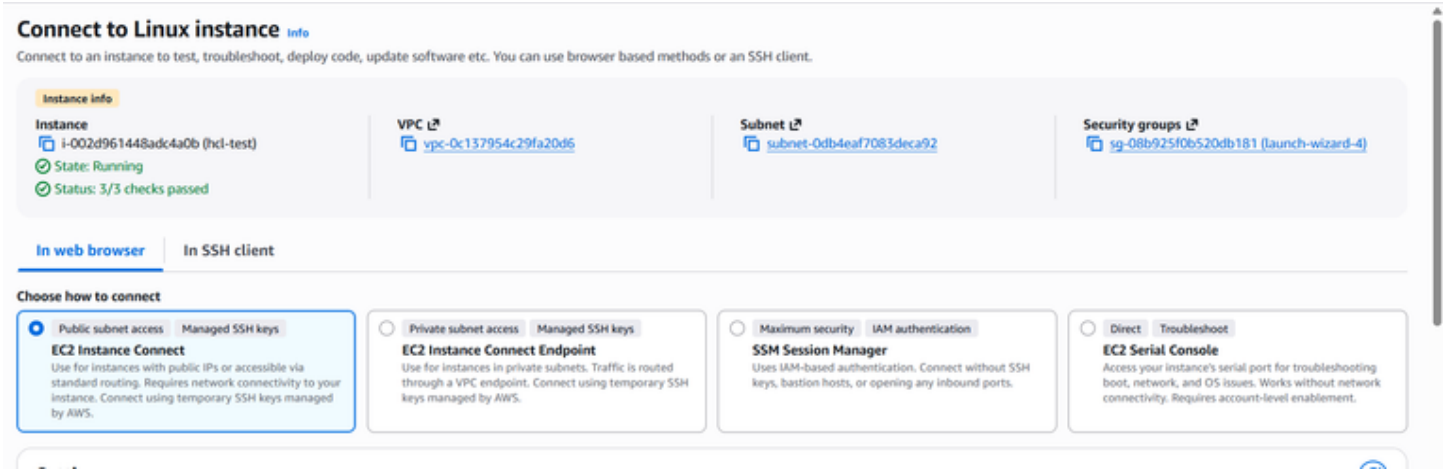
d. Explain how encryption at rest protects the stored data. **(1 mark)**
Encryption at rest protects stored data by encrypting it with encryption keys, preventing unauthorized users from reading the data without the required decryption key.

Question 11: EC2 Troubleshooting and CloudWatch (5 Marks)
A web application hosted on EC2 becomes slow, and users report connectivity problems.

Perform or document the following:

a. Check the EC2 instance state and status checks. **(1 mark)**



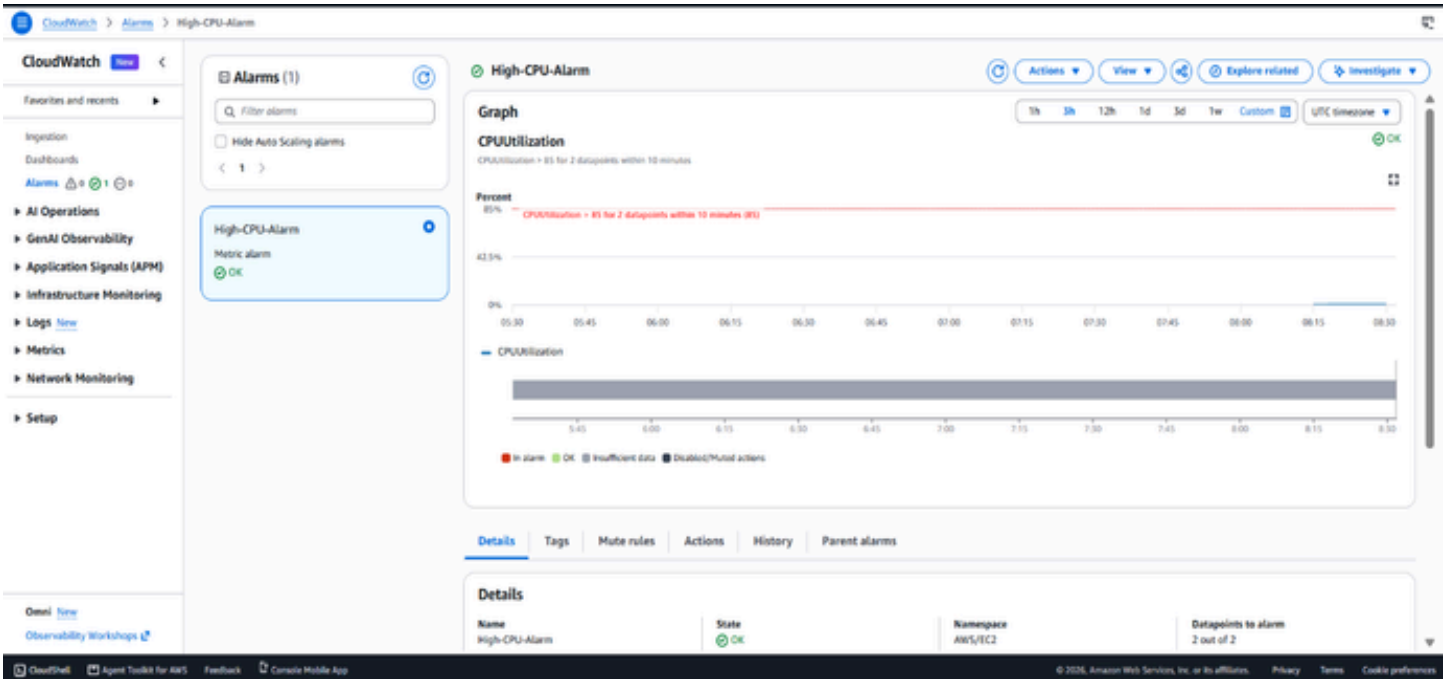


a. Explain how security groups and route tables can affect application connectivity. **(1 mark)**

Security groups control inbound and outbound traffic, while route tables determine traffic paths; incorrect rules or routes can prevent application connectivity.

a. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period. **(2 marks)**

In Amazon CloudWatch, select the EC2 CPU Utilization metric, create an alarm with a threshold of 85%, set the evaluation period and number of datapoints, and configure an SNS notification. The alarm triggers when CPU utilization exceeds 85% for the defined evaluation period.



a. Identify one method for investigating the root cause of high CPU utilization. **(1 mark)**

Use Amazon CloudWatch metrics and logs to identify CPU usage spikes and determine which process or application is consuming excessive CPU resources.

Question 12: Event-Driven Processing (5 Marks)

a. Identify a suitable service for executing the processing code. (1 Mark)

AWS Lambda is suitable for executing code to validate and process CSV files automatically without managing servers.

b. Explain how an S3 upload can trigger processing. (1 Mark)

Configure an S3 Event Notification for the s3:ObjectCreated:* event. When a CSV file is uploaded to the input bucket, Amazon S3 automatically triggers the Lambda function to process the file.

c. Explain the IAM permissions needed to read the input file and write the results. (1 Mark)

Attach an IAM execution role to the Lambda function with the following permissions:

- s3:GetObject to read CSV files from the input bucket.
- s3:PutObject to write processed results to the output bucket.
- CloudWatch Logs permissions to record execution logs.

Grant access only to the required bucket paths, following the principle of least privilege.

d. Explain how errors and processing logs can be captured. (1 Mark)

Use Amazon CloudWatch Logs to store Lambda execution logs and error messages. Configure CloudWatch alarms to monitor failures. An SQS dead-letter queue or Lambda asynchronous failure destination can capture failed processing events for later investigation and retry.

e. Explain how to prevent duplicate processing or recursive triggering. (1 Mark)

- Prevent duplicate processing: Use an idempotency mechanism, such as recording processed file keys in DynamoDB, to avoid processing the same file more than once.
- Prevent recursive triggering: Store output files in a separate S3 bucket or configure event notifications to trigger only for the input folder with a .csv suffix filter.

This ensures reliable processing and prevents the Lambda function from repeatedly triggering itself.

SECTION E: AWS STEP FUNCTIONS || Maximum Marks: 15

Question 13: State Machine Design (7 Marks)

A company needs to process sales files through the following stages:

1. Validate the input file.
2. Transform the data.
3. Calculate total sales by product.
4. Store the results in Amazon S3.
5. Send a completion notification.

Design an AWS Step Functions state machine to coordinate the workflow.

a. Identify suitable states for validation, processing, storage, and notification. **(2 marks)**

Suitable states for the workflow (2 Marks)

- Task states: Validate, transform, calculate total sales, store results, and send notification.
- Choice state: Checks whether the input file is valid.
- Fail state: Stops processing when validation fails.
- Succeed state: Ends the workflow successfully.

a. Explain how a Choice state can route invalid files to a failure path and valid files to processing. **(1 mark)**

The Choice state checks a Boolean field such as isValid.

- If isValid = true, the workflow proceeds to data transformation.
- If isValid = false, the workflow routes to a Fail state.

a. Explain how Retry and Catch can handle transient failures and permanent errors. **(2 marks)**

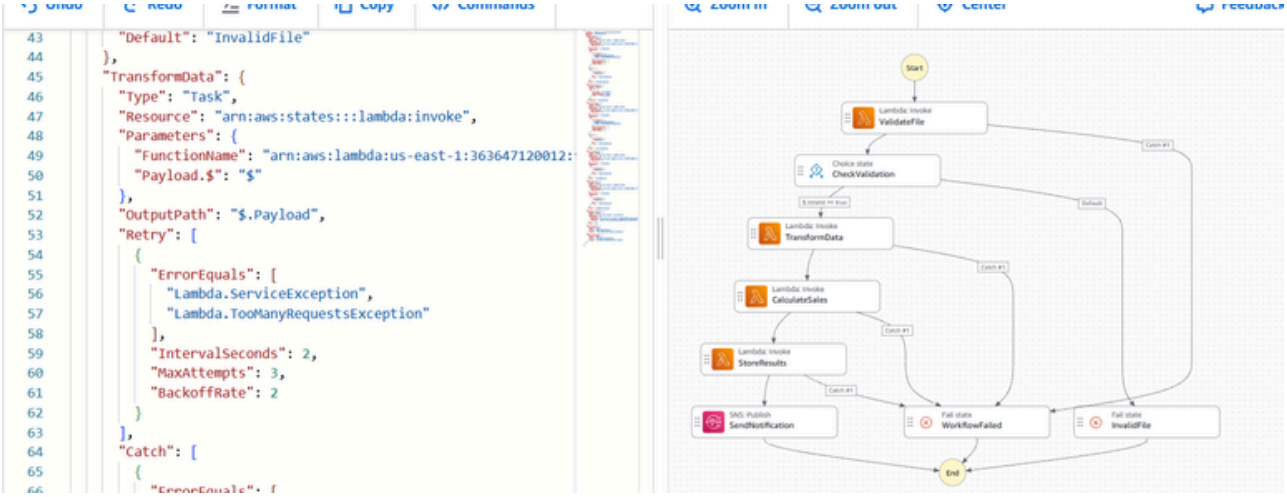
Retry automatically repeats a failed task for transient errors, such as temporary network issues. Catch handles persistent or permanent errors by routing execution to a failure-handling state or sending an error notification.

a. Explain how the state machine can invoke Lambda functions or other supported AWS services. **(1 mark)**

The state machine uses Task states with a Lambda function ARN or AWS service integration, along with appropriate IAM permissions, to invoke Lambda functions or other supported AWS services

a. Explain how execution history helps troubleshoot a failed workflow. **(1 mark)**

Execution history shows each state's input, output, status, and error details, helping identify where and why the workflow failed.



Question 14: Step Functions Implementation (8 Marks)

A data engineering team has developed three processing functions:

- ValidateFile
- TransformData
- LoadResults

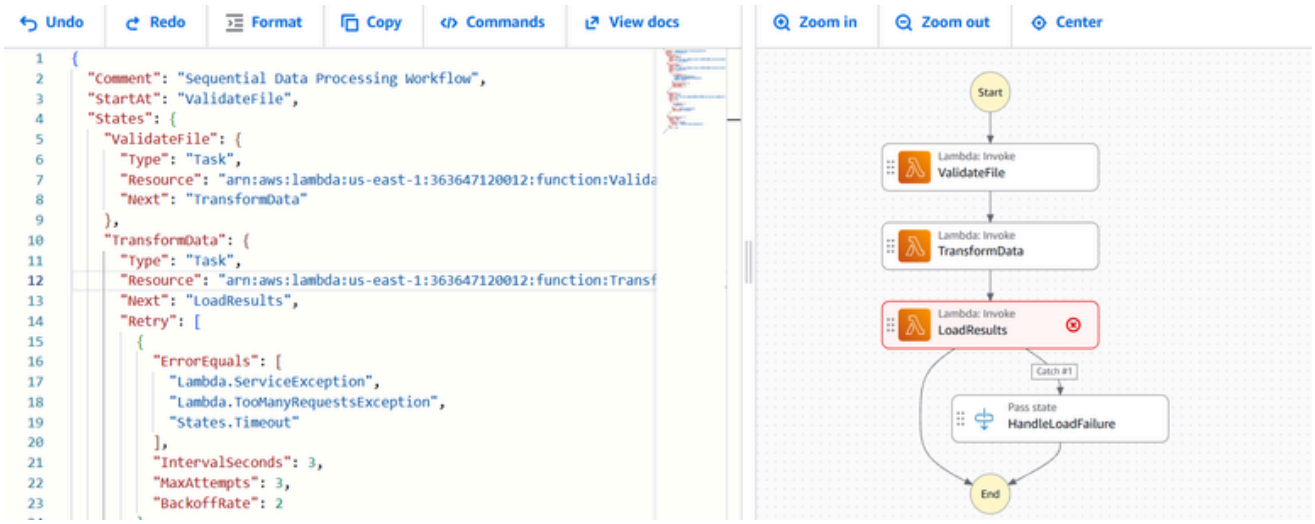
The functions must execute in sequence.

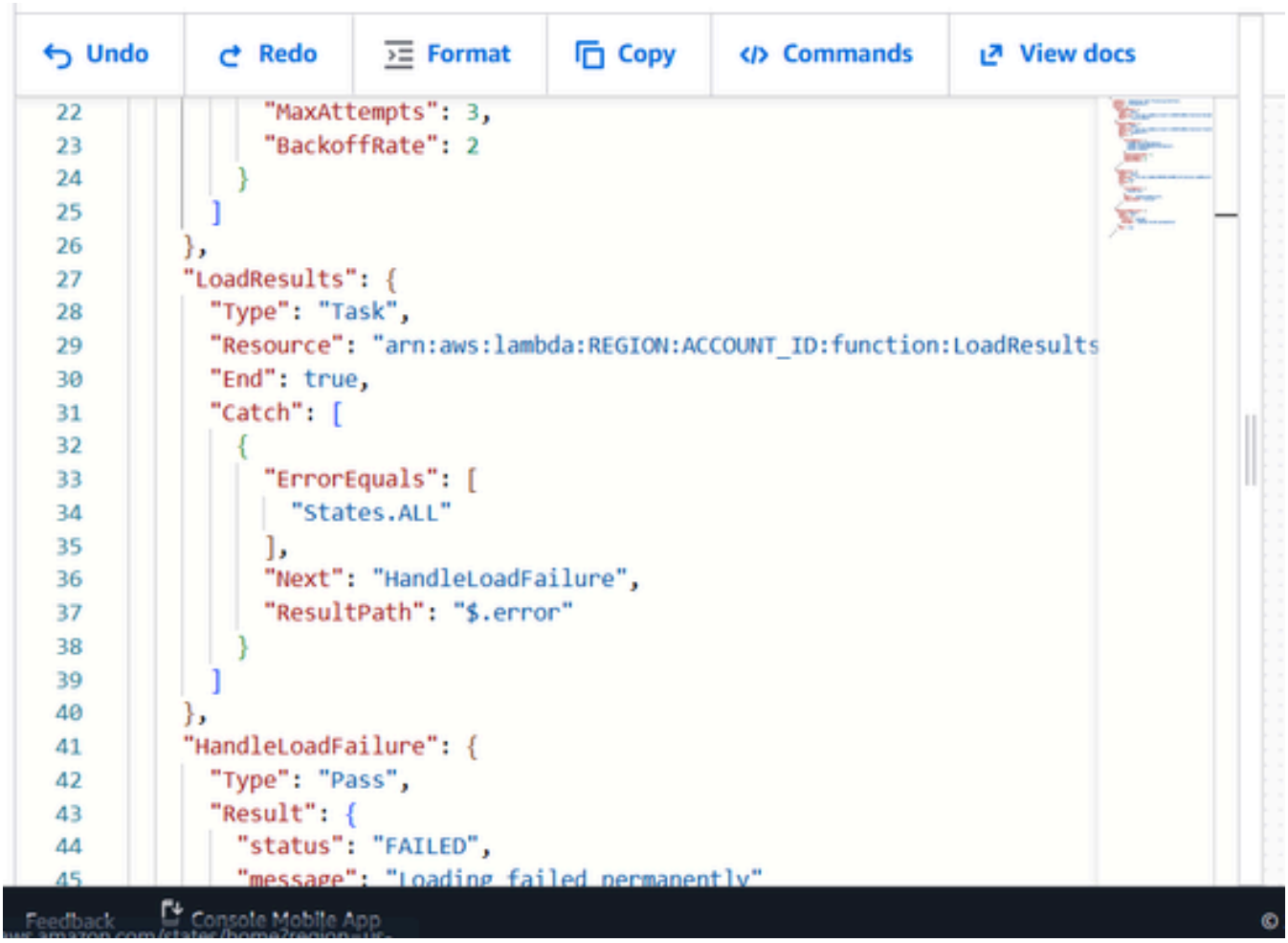
Requirements:

- If validation fails, the workflow must stop.
- If transformation fails temporarily, the workflow must retry.
- If loading fails permanently, the workflow must record the failure.

Tasks:

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary. **(3 marks)**





```
        ],
        "Next": "HandleLoadFailure",
        "ResultPath": "$.error"
      }
    ]
  },
  "HandleLoadFailure": {
    "Type": "Pass",
    "Result": {
      "status": "FAILED",
      "message": "Loading failed permanently"
    },
    "End": true
  }
}
```

b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts. (2 marks)

The Transform Data task can be configured with a Retry policy to handle temporary errors. The Error Equals field specifies error categories such as Lambda.ServiceException, Lambda.TooManyRequestsException, and States. Timeout. The Interval Seconds is set to 3 seconds, Max Attempts is set to 3 retries, and Backoff Rate is set to 2, which doubles the waiting time between retries. If the task continues to fail after all retry attempts, the workflow fails unless additional error handling is configured.

c. Configure a Catch transition for the loading task that routes to an error-handling state. **(1 mark)**

Configure a Catch block in the Load Results task with Error Equals: ["States. ALL"] and Next: "HandleLoadFailure". If loading fails, the workflow transitions to the HandleLoadFailure state, which records the failure status and ends the execution.

d. Explain how data from one task is passed to the next task. **(1 mark)**

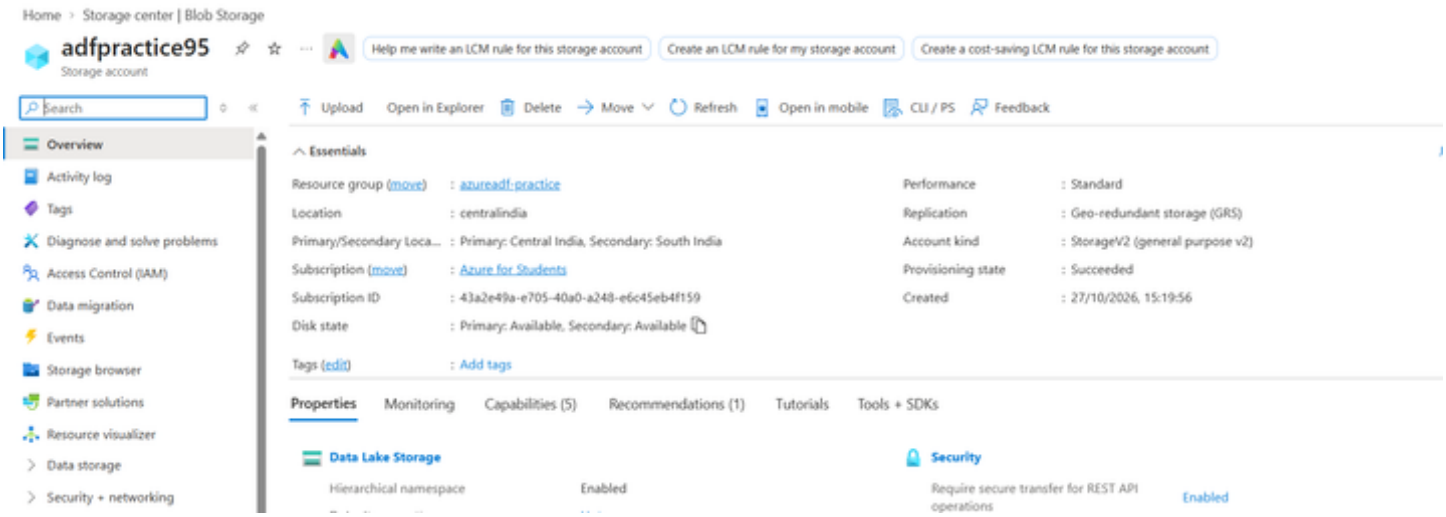
In AWS Step Functions, the output of one task becomes the input of the next task by default. For example, the output of ValidateFile is passed to TransformData, and the transformed output is passed to LoadResults. The InputPath, ResultPath, and OutputPath fields can be used to control how data is passed between tasks.

e. Explain how you would test successful and failed executions. **(1 mark)**

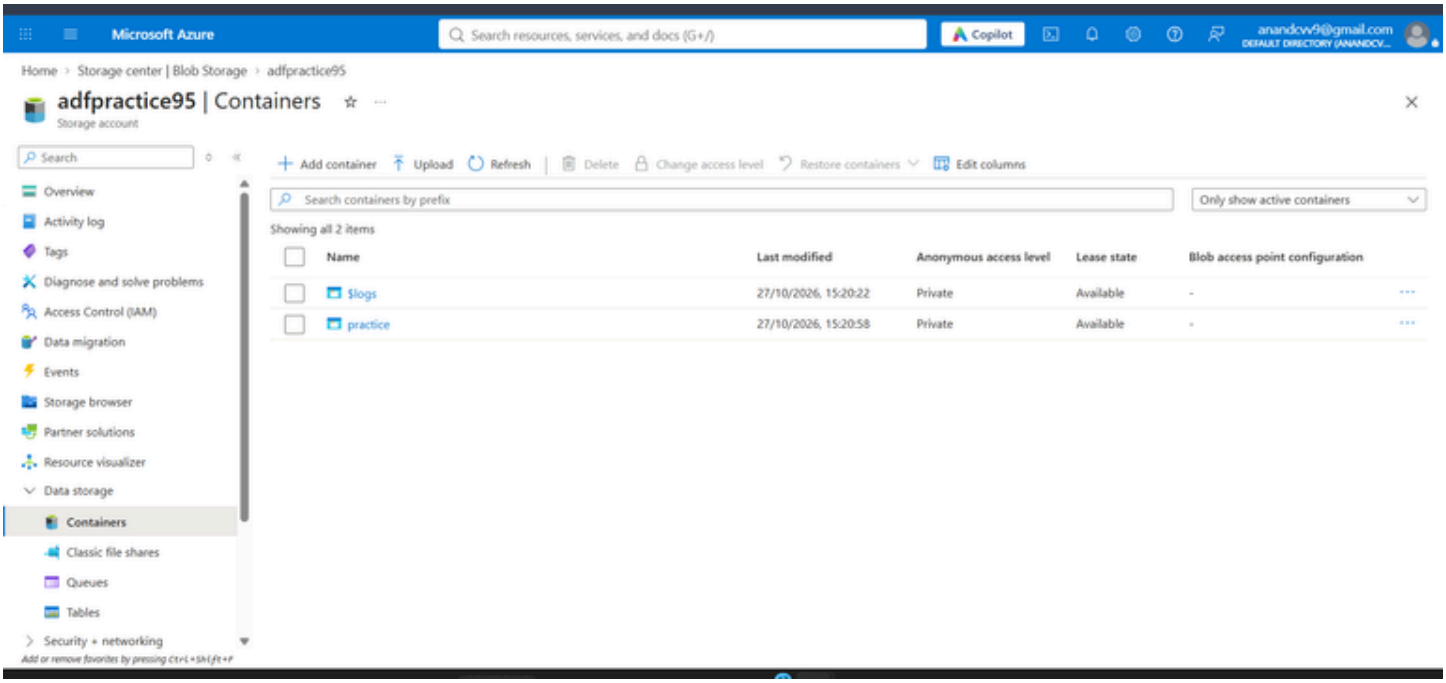
To test the workflow, run it from the AWS Step Functions console with valid input and verify that all three tasks execute successfully in sequence. For failure testing, simulate a validation error, a temporary transformation error, and a loading error. Verify that validation stops the workflow, transformation retries, and loading failure routes to the error-handling state.

Expected Output: Submit the JSON definition and explain the states, transitions, and error-handling strategy.

SECTION F: MICROSOFT AZURE || Maximum Marks: 15
Question 15: Azure Data Lake Storage and Security (5 Marks)
A company needs to store confidential business data in Azure for analytics.
Perform or document the following:
a. Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2. **(1 mark)**



b. Create a filesystem or container and upload a sample dataset. **(1 mark)**

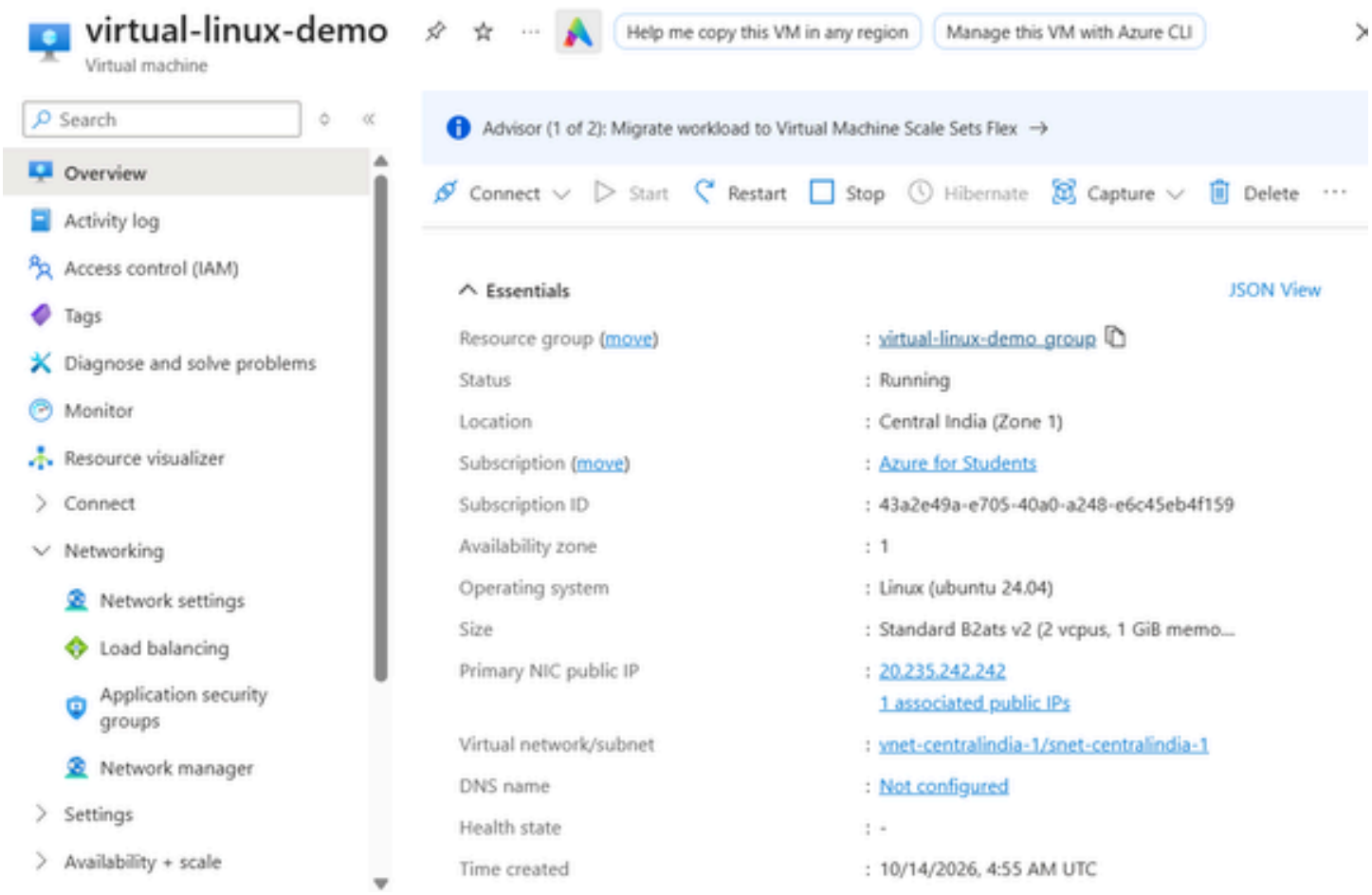


c. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data. **(2 marks)**
Microsoft Entra ID authenticates users and groups before they access Azure Data Lake Storage. Azure role-based access control (RBAC) assigns permissions based on users' roles. For example, Storage Blob Data Reader allows users to read data, while Storage Blob Data

Contributor allows them to read, write, and delete data. By assigning appropriate roles through Access Control (IAM), the company can ensure that only authorized users access confidential data**.**

d. Explain how you would verify that unauthorized users cannot access the dataset. **(1 mark)**
I would verify access by signing in with an unauthorized test user who has no assigned permissions and attempting to access or download the dataset. If Azure denies the request with a 403 Forbidden (Authorization Failure) error, it confirms that the user cannot access the data.

Question 16: Azure VM Troubleshooting (5 Marks)
A virtual machine is running in Azure, but administrators cannot connect to it.
Perform or document the following:
a. Verify the VM's provisioning and power states. **(1 mark)**



b. Explain how Network Security Group rules can prevent connectivity. **(1 mark)**
Network Security Group (NSG) rules control inbound and outbound network traffic using source IP addresses, destination ports, and protocols. Allow rules permit authorized traffic, while deny rules block unwanted connections, preventing unauthorized access to Azure resources.
c. Explain how subnet and routing configuration affect connectivity. **(1 mark)**
Subnet configuration divides an Azure virtual network into smaller networks and controls which resources can communicate with each other. Routing tables determine the path network

traffic takes between subnets and other networks. Incorrect subnet settings or routes can block connectivity or send traffic through the wrong network path.

d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet. **(2 marks)**
Use Azure Bastion to securely connect to the VM through the Azure Portal without exposing its public IP or opening SSH/RDP ports to the internet. Also, use Network Watcher to diagnose NSG rules, routing issues, and connectivity problems. This approach improves security while allowing effective troubleshooting.

Question 17: Azure Data Integration (5 Marks)
A company receives a CSV file containing daily customer transactions and needs to prepare it for analysis.
Design a solution using Azure services.
a. Select an appropriate service to orchestrate data movement. **(1 mark)**
Use Azure Data Factory (ADF) to orchestrate data movement. It allows us to create pipelines that automate the extraction, transformation, and loading of customer transaction data.

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2. **(1 mark)**
Create an Azure Data Factory pipeline with a Copy Data activity. Configure the CSV file as the source dataset and Azure Data Lake Storage Gen2 as the destination. Set up the linked services, configure the file path, and run the pipeline to copy the data into the data lake.

c. Describe how to organize files by ingestion date. **(1 mark)**
Store CSV files in date-based folders, such as transactions/YYYY-MM-DD/transactions.csv. This makes files easy to organize and find by ingestion date.

d. Explain how to handle malformed records and missing values. **(1 mark)**
Use Azure Data Factory Mapping Data Flows or Azure Databricks to validate and clean the CSV data. Handle missing values by filling defaults or rejecting incomplete records, and redirect malformed records to a separate error folder for further investigation.

e. Explain how to monitor pipeline execution and validate data completeness. **(1 mark)**
Use the Monitor section in Azure Data Factory to check pipeline status, execution time, and errors. Validate data completeness by comparing source and destination record counts and checking for missing or duplicate records. Re-run failed activities when necessary.